PiPER ボトル ピック＆プレイス 手順と流れ

リポジトリ piper-bottle-pp v0.1.0 ／ 1号機での立上げ 2026-10-28 ／ 別機体での再現性確認用

AgileX PiPER の手首に RGB-D カメラを付け、深度の点群だけでボトルのキャップ位置を求め、 キャップを横から挟んで2つの台の間を往復させる。制御は Jetson Orin 上の Python（pyAgxArm）だけで行い、ROS2・MoveIt・学習モデルは使わない。

1. 1号機での結果

2.2 mm
ハンドアイ校正のばらつき平均（最大 4.6 mm、17姿勢）

±5 mm 以内
校正で推定したマーカー位置とメジャー実測の差

207 mm
点群で求めたボトル高さ（実測 207 mm）

A↔B 反復
台A・B間の往復を繰り返し実行し成功

項目	内容
ロボット	AgileX PiPER（HW H-V1.2-1、FW S-V1.8-2）、標準グリッパ AgxGripper（最大 70 mm、把持力 0〜3 N）
制御PC	Jetson Orin 16GB、JetPack 6.2.1（L4T R36.4.7、Ubuntu 22.04.5、カーネル 5.15.148-tegra）
カメラ	Orbbec Dabai DC1（USB2.0、深度 640×400、カラー 640×480 MJPG）。グリッパ側面に上下逆に取付け（eye-in-hand）
対象	500 ml の不透明なボトル（満水で約 550 g）、キャップ直径 30 mm、高さ 207 mm
把持	キャップ上端から 10 mm 下を、水平から 30° 下向きのグリッパで左右から挟む。力 2 N
動作	観察姿勢 look で両台を撮影 → ボトルのある台からつかむ → 10 cm 持ち上げ → 反対の台へ置く → look で置き位置を確認（15 mm 以内で成功）

2. システム構成

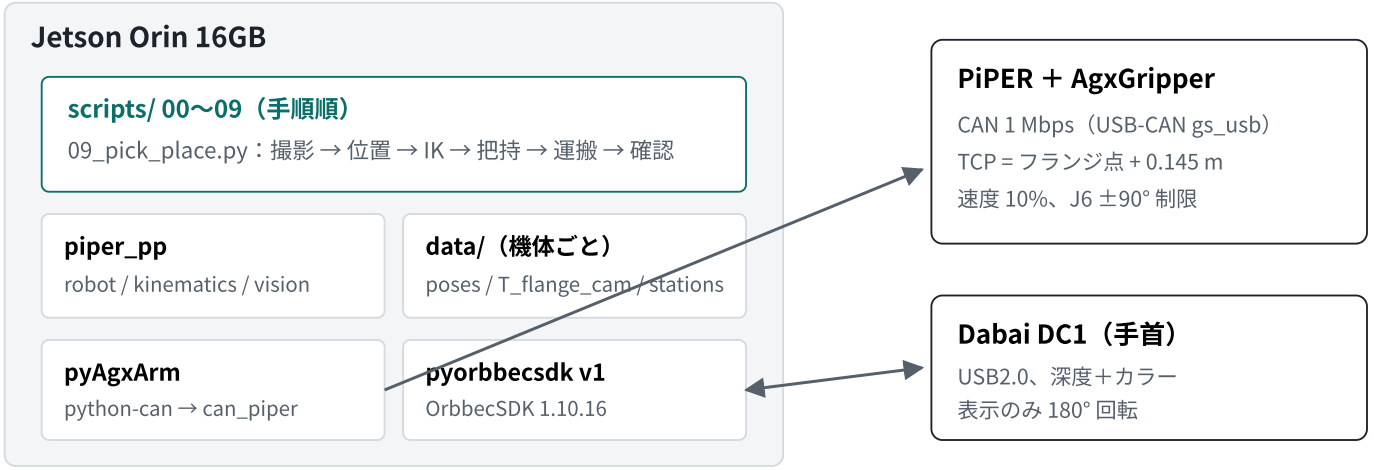


図1 構成。腕の指令とカメラ取得はすべて Jetson 上の1つの Python プロセスで行う（カメラは同時に1プロセスしか開けない）。

3. 全体の流れ

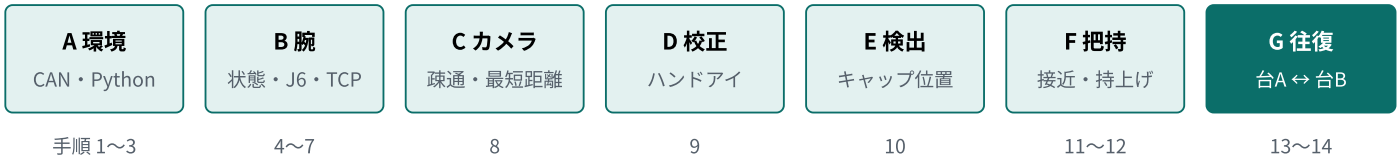


図2 手順の段階。各段階の合格基準を満たしてから次へ進む。前の段階に戻るのは、校正値（TCP・ハンドアイ）を変えたとき。

> 各手順の「1号機」は 2026-10-28 の実測値。別機体ではこの値に近いかを比べ、大きく違う場合はそこで止めて原因を調べる。

4. 手順

A 環境（Jetson）

1 USB-CAN ドライバ（gs_usb）

```
bash system/build_gs_usb.sh
```

理由	JetPack 6 の標準カーネルに gs_usb が無い。稼働カーネルと同じ版の gs_usb.c を kernel.org から取得し、外部モジュールとしてビルドする
合格	`lsmod | grep gs_usb` に出る。`ip -br link` に can1 などが出る（can0 は本体内蔵の mttcan）
注意	入れ先は `/lib/modules/$(uname -r)/updates/`（drivers/net/can/usb/ は存在しない）。JetPack を更新したら再ビルド。"tainting kernel" の通知は無害

2 CAN の名前と速度を固定

```
bash system/setup_can.sh
candump can_piper -n 20     # PiPER の電源 ON で 2A1〜2A8・251〜256 が流れる
```

合格	`ip -details link show can_piper` が ERROR-ACTIVE、bitrate 1000000。抜き差し後も自動で up
1号機	USB ポート 1-2.2:1.0。別のポートに挿したら setup_can.sh を再実行

3 Python 環境

```
bash system/install_python.sh && source ~/.bashrc
cd ~/piper-bottle-pp && pytest -q          # 実機なしの確認
```

内容	venv（~/venv_pp、system-site-packages 付き）、pyAgxArm、pyorbbecsdk v1 系（main ブランチ）、piper_pp（pip install -e .）
合格	pytest が 8 件すべて pass（逆運動学、ハンドアイ計算、キャップ位置推定を合成データで確認）
注意	Dabai DC1 は OpenNI 系のため pyorbbecsdk v2 系では認識しない。v1 系を使う

B 腕

4 接続と状態（動かさない）

```
python scripts/00_check.py
```

合格	FW 版が読める、関節角が更新される、arm_status 正常、全関節ドライバ異常なし
設定	FW S-V1.8 系以降は `config/robot.json` の firmware = "default"。古い版は v183 など
1号機	FW S-V1.8-2、受信約 2900 Hz。電源投入時 J6 = −121°、J4 = −102.6° で SDK 制限の外だった

5 J6 の原点確認（必要なら再校正）

```
python scripts/00g_range.py            # 脱力した J6 を手で両端まで回す → 中心を表示
python scripts/00h_calib_j6.py <中心>   # 中心が大きくずれていたときだけ（yes 入力で書込み）
```

合格	範囲の中心が 0° 付近（幅は約 340°）
1号機	中心が約 −288.6° ずれていた → 再校正後 −168.49〜+174.29°（中心 +2.9°、そのまま使用）
注意	SDK は範囲外の値を捨てるため表示が止まって見える。実値は `00f_raw_j6.py`（CAN 生データ ID 0x2A7）で見る。全軸一括の校正（255）は使わない

6 初回動作

```
python scripts/01_first_move.py
```

合格	手首0 → 全関節0 → 前傾 → 全関節0 がすべて到達、グリッパ 開 約0.06 m／閉 約0 m

7 TCP 長さの照合と look 姿勢

```
python scripts/02_jog.py     # ? でコマンド一覧。p は表示のみ（動かない）
```

TCP	ノギスでグリッパ取付け面から爪先端中央までを測り、**+50 mm** した値を `tcp_z` にする（SDK のフランジ点は取付け面より約 50 mm 手首側）。 折りたたみ姿勢（h）で J1 軸中心から先端までの水平距離と天板からの高さを実測し、表示の TCP x・z と ±5 mm で合えば OK
look	台A・B を置き、両方のボトルが爪にかからず上から見下ろせる姿勢にして `s look`。レンズからボトルまで 25 cm 以上（最短測距 189 mm）
1号機	実測 95 mm → tcp_z = 0.145 m。折りたたみ姿勢で表示 x 201／実測 200 mm、z 226／実測 227 mm。ベース上面が z = 0、木の台の上面 z = 21 mm

C カメラ

8 カメラ疎通と最短距離

```
python scripts/03_cam_check.py > /dev/null     # 内部パラメータと中央の深度
python scripts/04b_live_view.py > /dev/null    # ライブ表示（q で終了）
```

合格	fx > 0、深度が取れる。50 cm の対象が ±3% 程度。ボトルの深度が欠けずに1つの塊で見える
1号機	カラー fx = fy = 488.1、cx 314.5、cy 210.7。深度→カラー −29.8 mm。50 cm で 514 mm。最短 189 mm
注意	SDK のログは標準出力に出るので `> /dev/null`。カメラは1プロセスしか開けない（他が開いていると res-6 BUSY）。透明ボトルは胴の深度が欠ける

D 校正

9 ハンドアイ校正（eye-in-hand）

```
# ターミナル1（カメラ）                # ターミナル2（腕）
python scripts/05_handeye.py 51.0 > /dev/null     python scripts/02_jog.py
```

準備	`docs/aruco_id0_50mm_A4.pdf` を等倍印刷し、黒い正方形の一辺をノギスで測る（引数に mm で渡す）。台の上に平らに固定
操作	ターミナル2で姿勢を変え、静止してからターミナル1の画面で c。位置と手首の向き（±20°）を変えて 10〜15 姿勢。q で計算
合格	起動時の「姿勢角の解釈チェック」1 mm 未満。ばらつき平均 3 mm 以下。推定したマーカー中心がメジャー実測と ±5 mm
1号機	17 姿勢、ばらつき平均 2.2／最大 4.6 mm。カメラ位置（フランジ基準）[−80.6, 16.7, 41.2] mm。推定 (350.9, 1.5, 21.8)／実測 (355, 3, 20) mm
注意	OpenCV 5 系には calibrateHandEye が無いため自前実装（Park-Martin 法 + 最小二乗）。爪がマーカーを隠すと検出しない

E 検出

10 キャップ位置の推定（学習なし）

```
python scripts/02_jog.py      # go look → q
python scripts/06_locate.py > /dev/null       # 確認画像 data/locate_result.jpg
```

方法	深度8枚の中央値 → ハンドアイでロボット座標 → 置き面 +30 mm より上の点 → 最上部（98%点）をキャップ上端 → 手前半分しか見えないキャップに半径 15 mm の円を当てはめて中心
合格	ボトル高さが実測 ±10 mm、2回の差 1 mm 程度、キャップ位置がメジャー実測 ±5 mm
1号機	見下ろし姿勢で高さ 207 mm（実測 207 mm）。ほぼ水平に見る姿勢では上端が +6 mm 高く出た → look は見下ろす向きに
注意	置き面の高さは自動判定せず固定値（`surface_z`）を使う（浅い角度で見る面は点がまばらで誤判定した）

F 把持

11 接近試験（つかまない）

```
python scripts/07_approach.py X Y Z --dry    # 手順10のキャップ上端 [mm]。関節角を表示するだけ
python scripts/07_approach.py X Y Z          # pre（手前6 cm）→ near（手前2 cm）で停止して目視
```

合格	near でキャップが2本の爪の中央にある（左右・上下とも数 mm 以内）

12 把持と 1 cm 持上げ

```
python scripts/08_pick_test.py X Y Z --down 10 --force 2
```

合格	閉じた幅が 5 mm 以上（0 なら空振り）、持ち上げて3秒保持後も幅が変わらない、ボトルが浮く
1号機	満水 500 ml（約 550 g）を力 2 N で持ち上げ成功

G 往復

13 台A・B の登録

```
python scripts/09_pick_place.py --setup A    # ボトルを台Aに置いて
python scripts/09_pick_place.py --setup B    # ボトルを台Bに置いて
```

内容	look から半径 30 cm 内でボトルを探し、キャップ上端を `data/stations.json` に記録。往復中はこの位置の半径 6 cm 内だけを探す

14 往復の実行

```
python scripts/09_pick_place.py 1            # 各段階で Enter 待ち
python scripts/09_pick_place.py --set-advance B 10   # 台Bで浅くつかむ場合（放射方向に +10 mm）
python scripts/09_pick_place.py 10 --auto    # 慣れたら連続
```

合格	置いた後の look で、キャップ位置が登録位置から 15 mm 以内（「成功」）。失敗するとその場で止まる
記録	`data/pp_trials.csv`：時刻、移動元・先、キャップ位置、閉じ幅、持上げ後の幅、結果、置き位置のずれ、サイクル時間
1号機	台Bで爪が浅く失敗 → 台Bに前進補正を設定して解消。A↔B を繰り返し実行できた

5. 1サイクルの動き

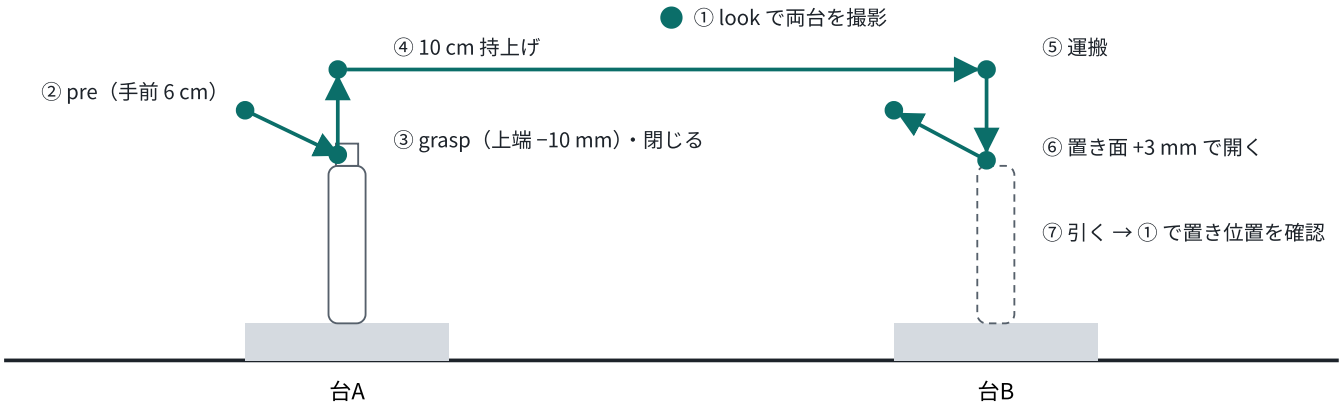


図3 台A → 台B の1サイクル（側面のイメージ）。グリッパは水平から 30° 下向きのまま、J1 軸から見て放射方向に近づく。台B → 台A も同じ。

設定（config/robot.json）	値	意味
pick.tilt_deg	30	グリッパの下向き角度（水平から）
pick.grip_down	0.010 m	キャップ上端から挟む位置までの距離
pick.pre	0.060 m	把持点の手前に引く距離（ツール軸方向）
pick.lift	0.100 m	持ち上げ高さ
pick.place_gap	0.003 m	置くときに置き面より少し上で放す
pick.force_n	2.0 N	把持力（上限 3 N）
pick.search_radius	0.06 m	往復中、登録位置の周りで探す半径
pick.success_err_mm	15 mm	置き位置のずれの合格線
surface_z	0.021 m	台の上面の高さ（両台で同じ前提）
tcp_z	0.145 m	SDK フランジ点から爪先端中央まで

6. 押さえておく性質

性質	内容と使い方
グリッパの向き	J4 = 0 のとき、J2+J3+J5 = 5° で水平、95° で真下。全関節0でグリッパは水平から 5° 上向き。逆運動学はこの関係で J5 を決め、J2・J3 を解く（J1 = atan2(y, x)）
真下向きの限界	J5 の SDK 制限 ±70° のため、真下向きは TCP 高さ約 0.08 m 以下でしか成立しない。キャップ高さ（約 0.22 m）には上から届かないので、斜め 30° から挟む
姿勢角の順序	SDK の [roll, pitch, yaw] は scipy の `Rotation.from_euler("xyz", …)`（= Rz·Ry·Rx）で一致。05_handeye.py が起動時に自己検証する
順運動学	pyAgxArm の `fk()` は CAN 接続なしで計算できる。逆運動学とテストはこれを使うので実機と一致する

カメラ画像の向き	上下逆の取付けのため表示だけ 180° 回転（`camera_display_rotate_180`）。計算には元画像を使う

7. 起きた問題と対処

現象	原因	対処
USB-CAN を挿しても CAN が出ない	JetPack 6 に gs_usb が無い	system/build_gs_usb.sh でビルド（updates/ に入れる）
J6 が −117° で stall・過電流、以降の指令が通らない	J6 原点が約 −288.6° ずれていた。SDK 表示は範囲外で固着	CAN 生データで範囲を測り、中心で calibrate_joint(6)、clear_joint_error(6)
傾けると TCP 高さが表示と 44 mm 違う	SDK のフランジ点は取付け面より約 50 mm 手首側	tcp_z = ノギス実測 + 50 mm
真下向きの直線移動がタイムアウト	J5 が制限 70° に到達	J5 に余裕のある姿勢で作り直す。把持は斜めから
カメラが開けない（res-6）	ライブ表示など別プロセスが使用中	他のカメラスクリプトを閉じる
置き面の高さを誤判定（193／177 mm）	浅い角度で見る面は点がまばら	surface_z を固定値で与える
キャップ上端が +6 mm 高い	ほぼ水平に見ると輪郭に背景との混合画素が出る	look を見下ろす向きに変更
キャップ中心が手前に寄る	キャップの手前半分しか見えない	半径 15 mm の円を当てはめる
台Bで爪が浅く失敗	台の位置により把持点の奥行きが不足	--set-advance B で放射方向に前進補正
保存姿勢 down が誤った値で上書き	未特定（同名で保存した可能性）	保存名を確認してから s。go の前に p で確認

8. 再現性確認の記録表（2号機用）

各手順の結果を記入する。1号機と大きく違う項目があれば、そこで止めて原因を調べる。

項目	1号機（2026-10-28）	2号機	判定
FW／HW	S-V1.8-2 ／ H-V1.2-1		
USB ポート	1-2.2:1.0		
J6 範囲・中心	−168.5〜+174.3°（+2.9°）※再校正後		
電源投入時の制限外の関節	J6 −121°、J4 −102.6°		
TCP 長さ（実測＋50）	95 + 50 = 145 mm		

TCP 照合（x／z）	表示 201／226、実測 200／227 mm		
カメラ内部パラメータ	fx 488.1、cx 314.5、cy 210.7		
最短測距	189 mm		
ハンドアイ ばらつき	平均 2.2／最大 4.6 mm（17姿勢）		
カメラ位置（フランジ基準）	[−80.6, 16.7, 41.2] mm		
マーカー 推定−実測	差 4／1.5／1.8 mm		
ボトル高さ 推定／実測	207／207 mm		
接近（near）の爪の中央	中央		
1 cm 持上げ	成功（2 N）		
前進補正	台Bのみ設定		
往復 成功／試行	繰り返し成功	／	
置き位置のずれ 平均／最大	pp_trials.csv 参照		
サイクル時間	pp_trials.csv 参照		

9. ファイル一覧

ファイル	用途	腕
system/build_gs_usb.sh	gs_usb のビルドと組込み、起動時ロード	—
system/setup_can.sh	USB-CAN を探して udev ルール作成、can_piper 1 Mbps	—
system/install_python.sh	venv、pyAgxArm、pyorbbecsdk v1、piper_pp	—
scripts/00_check.py	FW・関節角・状態・ドライバ異常の表示	動かさない
scripts/00d_disable.py	脱力（全関節なら手で支える）	脱力
scripts/00f_raw_j6.py	CAN 生データで J5・J6 を表示	動かさない
scripts/00g_range.py	J6 の手回し範囲と中心	動かさない
scripts/00h_calib_j6.py	J6 原点の書き直し（yes 確認）	原点書込み
scripts/01_first_move.py	初回動作 4 姿勢＋グリッパ	動かす
scripts/02_jog.py	対話ジョグ、直線移動、グリッパ、姿勢保存	動かす
scripts/03_cam_check.py	カメラのパラメータ、中央深度、最短距離	—
scripts/04_snap.py ／ 04b_live_view.py	画像保存 ／ ライブ表示	—
scripts/05_make_marker.py ／ 05_handeye.py	ArUco 画像作成 ／ ハンドアイ校正	読むだけ
scripts/06_locate.py	キャップ位置の推定と確認画像	読むだけ

scripts/07_approach.py	キャップ手前まで接近（--dry あり）	動かす
scripts/08_pick_test.py	把持と 1 cm 持上げ	動かす
scripts/09_pick_place.py	台A・B の登録、前進補正、往復	動かす
tests/test_offline.py	実機なしの確認（pytest）	—

10. ライセンスと公開前の確認

> このリポジトリのライセンスは未定（社内で決定）。公開前に社内承認を得る。リポジトリには機体の IP アドレス、カメラのシリアル番号、data/ の中身を入れない。

依存	ライセンス	扱い
pyAgxArm	LGPL-3.0	同梱せず pip で入れる
pyorbbecsdk	Apache-2.0	同梱せずソースからビルド
gs_usb.c（Linux）	GPL-2.0	同梱せず、ビルドスクリプトのみ
Ultralytics YOLO	AGPL-3.0	使っていない（今後使う場合は要確認）